分布式系统第八章容错复习大纲

根据您提供的文档，我将为您整理第八章“容错”的详细复习大纲。本章是重难点，内容涉及容错的基本概念、进程容错、可靠通信、分布式提交和系统恢复等。大纲基于文档4（第八章容错课件）为主，并参考其他文档的相关部分。我会将英文术语翻译成中文，详细解释重点内容，概括非重点，并标记关键内容在PPT中的位置（使用节标题如“§8.1”作为参考，因为文档中没有明确页码数字）。结构上，我将分节展开，每个部分包括核心概念、关键点、例子和PPT参考。

第八章容错概述

- **重点内容** ：容错是分布式系统的核心特性，确保系统在部分组件失效时仍能正确运行。本章涵盖容错定义、失效模型、冗余技术、共识协议、可靠通信和恢复机制。
- **PPT参考** ：文档4开头部分（标题“第8讲容错”）。

---

§8.1 基本概念

PPT参考 ：文档4的§8.1节。

这部分介绍容错的基础，包括定义、可依赖性要求和失效模型。

1. 容错定义（Fault Tolerance）

- **核心概念** ：容错指系统在组件失效时，仍能维持整体功能，不会严重受影响。系统可进行恢复并继续任务处理。
- **可依赖性（Dependability）** ：容错系统的目标，包括：
- **可用性（Availability）** ：系统准备就绪可供使用的比例，计算公式：A = MTTF / (MTTF + MTTR)，其中MTTF是平均失效时间，MTTR是平均恢复时间。
- **可靠性（Reliability）** ：系统在给定时间内无故障连续运行的能力，用MTTF衡量。
- **安全性（Safety）** ：灾难性事件概率极低。
- **可维护性（Maintainability）** ：故障系统修复的难易程度。
- **例子** ：银行系统要求高可用性（如99.999%在线时间），而航天系统更注重可靠性（无故障运行）。

2. 失效与故障模型

- **失效（Failure）** ：组件未按规格运行（如程序崩溃）。
- **错误（Error）** ：可能导致失效的组件状态（如编程bug）。
- **故障（Fault）** ：错误的原因（如粗心程序员），分为：
- 暂时故障（Transient）：偶然发生。
- 间歇故障（Intermittent）：周期性发生。
- 持久故障（Permanent）：需修复才能消除。
- **失效模型（故障模式）** ：
- 崩溃性故障（Crash Failure）：服务器停机，但停机前工作正常。
- 遗漏性故障（Omission Failure）：包括接收遗漏和发送遗漏。
- 定时故障（Timing Failure）：响应超时。
- 响应故障（Response Failure）：值错误或状态转换错误。
- 随意性故障（Arbitrary Failure/Byzantine Failure）：恶意或非恶意任意行为。
- **重点** ：拜占庭失效是最复杂的模型，需特殊协议处理（如BFT）。
- **PPT图片参考** ：文档4中有图表示失效模型（如表格），可参考“失效模型”部分。

3. 失效探测与处理

- **失效探测** ：基于超时机制，但在异步系统中难以可靠检测（如FLP不可能定理）。
- **处理方式** ：
- 故障预防（Fault Prevention）：防止故障发生。
- 故障容错（Fault Tolerance）：屏蔽故障影响。
- 故障移除（Fault Removal）：减少故障存在。

- 故障预测（Fault Forecasting）：估计故障后果。

---

§8.2 进程容错/恢复

PPT参考 ：文档4的§8.2节。

这部分讨论通过进程组和复制实现容错，以及共识协议。

1. 容错进程组与进程复制

- **核心概念** ：将多个进程组织成组，作为单一抽象处理。通过复制屏蔽失效。
- **容错度** ：K-容错组可屏蔽最多K个成员失效。
- 对于崩溃性失效：需K+1个成员。
- 对于拜占庭失效：需2K+1个成员（多数表决）。
- **例子** ：数据库副本系统，使用主备复制（Primary-based）或全复制写（Replicated-write）来确保一致性。
- **PPT图片参考** ：文档4中有图展示进程组通信（如平等组与等级组）。

2. 分布式共识（Consensus）

- **定义** ：非故障进程在有限步骤内就某个值达成一致（如选举或事务提交）。
- **系统模型假设** ：
- 同步系统：有时钟约束，可可靠检测失效。
- 异步系统：无时间约束，共识难达成（FLP不可能定理）。
- 部分同步系统：折中方案，常用超时机制。
- **共识协议** ：
- **Paxos协议** ：用于非恶意失效场景，基于两阶段协议（提案和接受），确保多数同意。

  ***例子** ：在分布式事务中，Paxos用于决定是否提交操作。
- **拜占庭容错协议（BFT）** ：处理恶意失效，如PBFT协议，需3f+1个节点容忍f个故障。

  ***例子** ：区块链系统使用BFT变种（如实用拜占庭容错）。
- **重点** ：共识是容错的基础，用于状态机复制和分布式事务。
- **PPT图片参考** ：文档4中有图展示共识协议分类和BFT过程。

---

§8.3 可靠C-S通信

PPT参考 ：文档4的§8.3节。

这部分关注如何实现可靠的客户端-服务器通信，包括RPC和组通信。

1. 可靠RPC（远程过程调用）

- **错误类型及处理** ：
- 错误1：客户端无法定位服务器 → 返回错误信息。
- 错误2：请求消息丢失 → 设置超时重发。

- 错误3：服务器宕机 → 区分执行前后宕机，使用至少一次（at-least-once）或至多一次（at-most-once）语义。
    - **例子** ：银行转账操作需幂等性（重复执行结果相同），以避免重复扣款。
- 错误4：应答丢失 → 设计幂等操作或使用序列号去重。
- 错误5：客户端宕机 → 处理孤儿计算，如通过租约机制（Lease）自动终止任务。
- **重点** ：精确一次语义（Exactly-once）在分布式系统中不可能完全保证，需权衡。
- **PPT图片参考** ：文档4中有图展示RPC错误场景（如服务器崩溃状态）。

2. 可靠组通信（多播）

- **原子多播（Atomic Multicast）** ：消息要么交付给所有进程，要么不给任何（全有或全无）。
- **消息顺序** ：
- FIFO顺序：同一进程消息按发送顺序交付。
- 因果顺序：有因果关系的消息按顺序交付（用向量时钟实现）。
- 全序（Total Order）：所有进程按相同顺序交付消息（用于状态机复制）。
- **虚拟同步（Virtual Synchrony）** ：确保消息交付与组成员变化同步，避免部分节点状态不一致。
- **例子** ：在组视图变化时（如节点加入/退出），消息按一致视图交付。
- **实现技术** ：分层多播（避免Ack内爆）、Gossip多播（概率性最终交付）。
- **PPT图片参考** ：文档4中有图展示虚拟同步和多播排序表格。

---

§8.4 分布式提交

PPT参考 ：文档4的§8.4节。

这部分讲解分布式事务的原子提交协议，如两阶段提交（2PC）和三阶段提交（3PC）。

1. 两阶段提交协议（2PC）

- **流程** ：

1. 协调者（Coordinator）向参与者（Participants）发送VOTE_REQUEST。
2. 参与者回复VOTE_COMMIT或VOTE_ABORT。
3. 如果所有参与者同意，协调者发送GLOBAL_COMMIT；否则发送GLOBAL_ABORT。
4. 参与者根据最终消息提交或中止。

- **失效处理** ：
- 参与者失效：在INIT状态可单方面中止；在READY状态可能阻塞（需等待协调者恢复）。
- 协调者失效：可能导致整个协议阻塞，需日志恢复或选举新协调者。
- **缺点** ：是阻塞协议（非完全非阻塞），可能无限期等待。
- **例子** ：分布式数据库事务，2PC确保所有节点要么全部提交，要么全部回滚。
- **PPT图片参考** ：文档4中有图展示2PC状态机。

2. 三阶段提交协议（3PC）

- **改进** ：增加预提交阶段（PRECOMMIT），减少阻塞可能。
- **流程** ：在2PC基础上，插入预提交状态，使参与者能在协调者失效时通过协商决定提交或中止。
- **优点** ：非阻塞性更好，但复杂度高。
- **重点** ：3PC通过额外阶段提高容错，但仍非完美。
- **PPT图片参考** ：文档4中有图对比2PC和3PC状态机。

---

§8.5 系统恢复

PPT参考 ：文档4的§8.5节。

这部分介绍系统失效后的恢复机制，如检查点和消息日志。

1. 检查点机制（Checkpointing）

- **概念** ：定期保存系统状态快照（Snapshot），用于回滚恢复。
- **恢复线（Recovery Line）** ：最近一致的全局检查点，是回滚目标。
- **实现方式** ：
- 协调检查点（Coordinated Checkpointing）：全局协调打检查点，但可能阻塞。
- 独立检查点（Independent Checkpointing）：各进程独立打检查点，但可能多米诺效应（级联回滚）。
- **例子** ：分布式计算任务定期打检查点，故障时从最近点恢复，避免重头计算。
- **PPT图片参考** ：文档4中有图展示检查点和恢复线。

2. 消息日志（Message Logging）

- **概念** ：记录非确定性事件（如消息接收），结合检查点实现确定性重放。
- **一致性策略** ：
- 悲观日志：消息在发送前必须持久化，避免孤儿进程。
- 乐观日志：允许快速执行，恢复时回滚孤儿进程。
- **孤儿进程（Orphan Process）** ：依赖未记录消息的进程，需通过日志避免。
- **重点** ：消息日志确保恢复后状态一致，是容错关键。
- **PPT图片参考** ：文档4中有图展示消息日志和孤儿进程场景。

3. 面向恢复的计算（Recovery-Oriented Computing）

- **理念** ：设计系统组件分离，使恢复局部化，快速重启部分系统。
- **例子** ：微服务架构中，单个服务故障不影响整体。

本章总结与重点标记

- **重难点** ：
- 共识协议（Paxos、BFT）和失效模型（§8.2）。
- 可靠RPC的错误处理（§8.3）。
- 分布式提交协议（2PC和3PC）的阻塞问题（§8.4）。
- 检查点和消息日志的恢复机制（§8.5）。
- **关键页码参考** ：全部内容基于文档4，节标题如§8.1至§8.5对应PPT幻灯片。建议您结合文档4的原文复习，重点关注图表和例子。
- **复习建议** ：多练习文档3中的试题（如两阶段提交和共识问题），以巩固理解。

如果您需要针对特定问题或例子进一步解释，请随时告诉我！